CSE_3523

Segment-7 (Part-1)
Microcontroller

Microprocessor, microcontroller and embedded system

Course code: CSE-3523
Credit Hour: 3

Md. Mujibur Rahman Maruf

Lecturer

Dept. of CSE,IIUC

Mujiburmaruf.cuet17@gmail.com

International Islamic University Chittagong(IIUC)

Introduction to the 8051 Microcontroller

What is a Microcontroller?

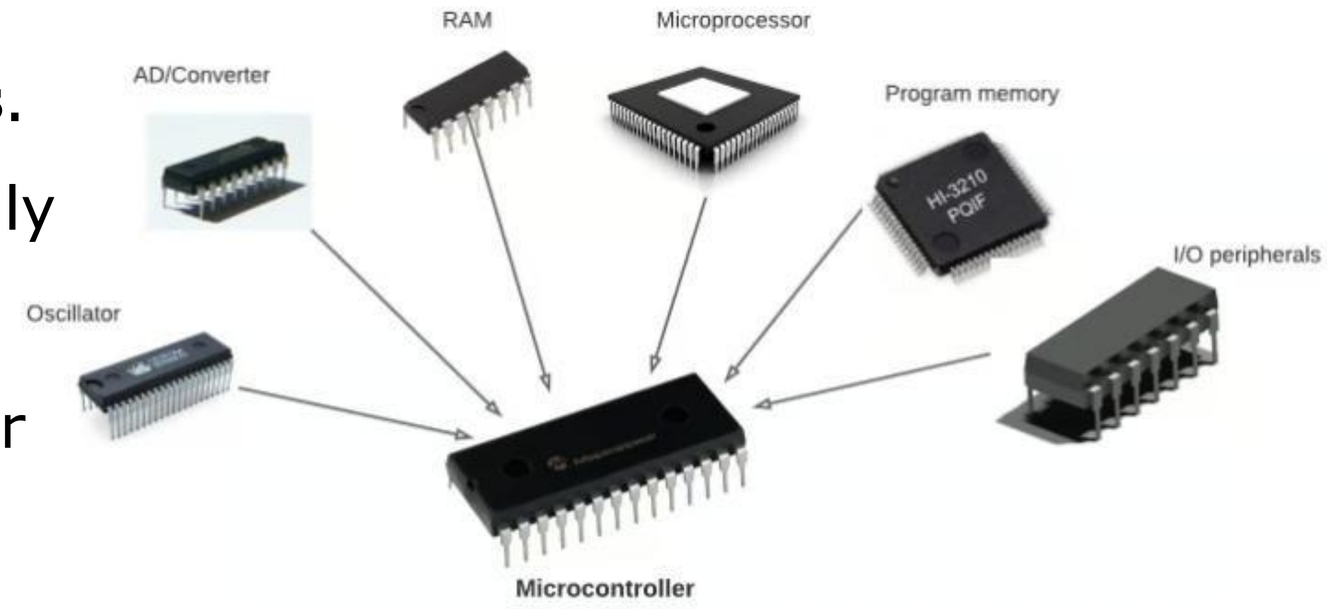
- A microcontroller is a **computer on a single chip**.
- It includes a **CPU, RAM, ROM, I/O ports, timers, and serial communication** interfaces—all built-in.
- Designed for **embedded applications** (washing machines, cameras, robotics).

Why 8051?

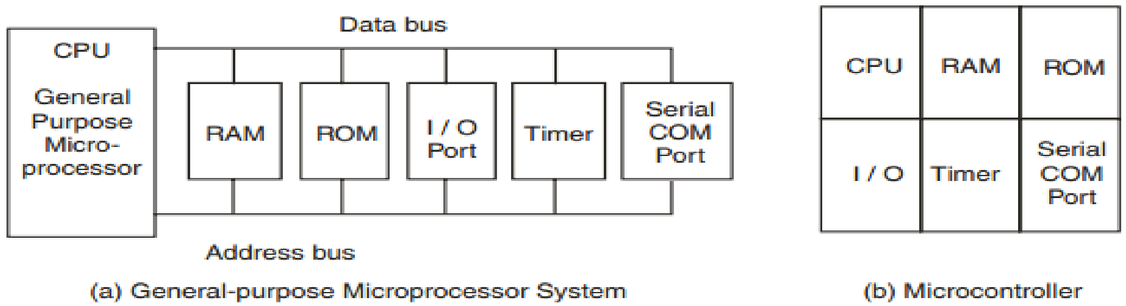
Developed by **Intel in the late 1970s**.

Most popular 8-bit microcontroller family (**MCS-51 series**).

Dominates ~40% of the microcontroller market.



Microprocessor Vs Microcontroller



(a) General-purpose Microprocessor System (b) Microcontroller

Fig. 1.1 Structure of microprocessor and microcontroller

Aspect	Microprocessor	Microcontroller (8051)
Integration Level	CPU only – external components needed (RAM, ROM, I/O, etc.)	CPU + RAM + ROM + I/O + Timers + Serial Port (on-chip)
System Design	Needs external interfacing for full functionality	All essential components are embedded in one chip
Memory	External RAM and ROM must be interfaced	128B RAM and 4KB ROM are inbuilt
Input/Output Ports	Must interface external devices like 8255 for I/O	Four 8-bit I/O ports available internally
Timers and Serial	External chips required (e.g., 8253 for timer, USART for serial)	Built-in timers and serial communication pins (T0, T1, TXD, RXD)
Cost and Size	Higher cost and larger board size	Lower cost, compact size ("System on Chip")
Application Suitability	Suitable for complex, high-processing systems	Best for small, real-time control applications
Example	Intel 8085, 8086	Intel 8051

Application of 8051

Energy Management

- Used in energy meters for homes and industries
- Enables precise energy consumption measurement
- Enhances smart meter performance

Touch Screens

- Integrated into portable devices (e.g., phones, media players, gaming devices)
- Many microcontroller brands include touch sensing
- Enables interactive and responsive UIs

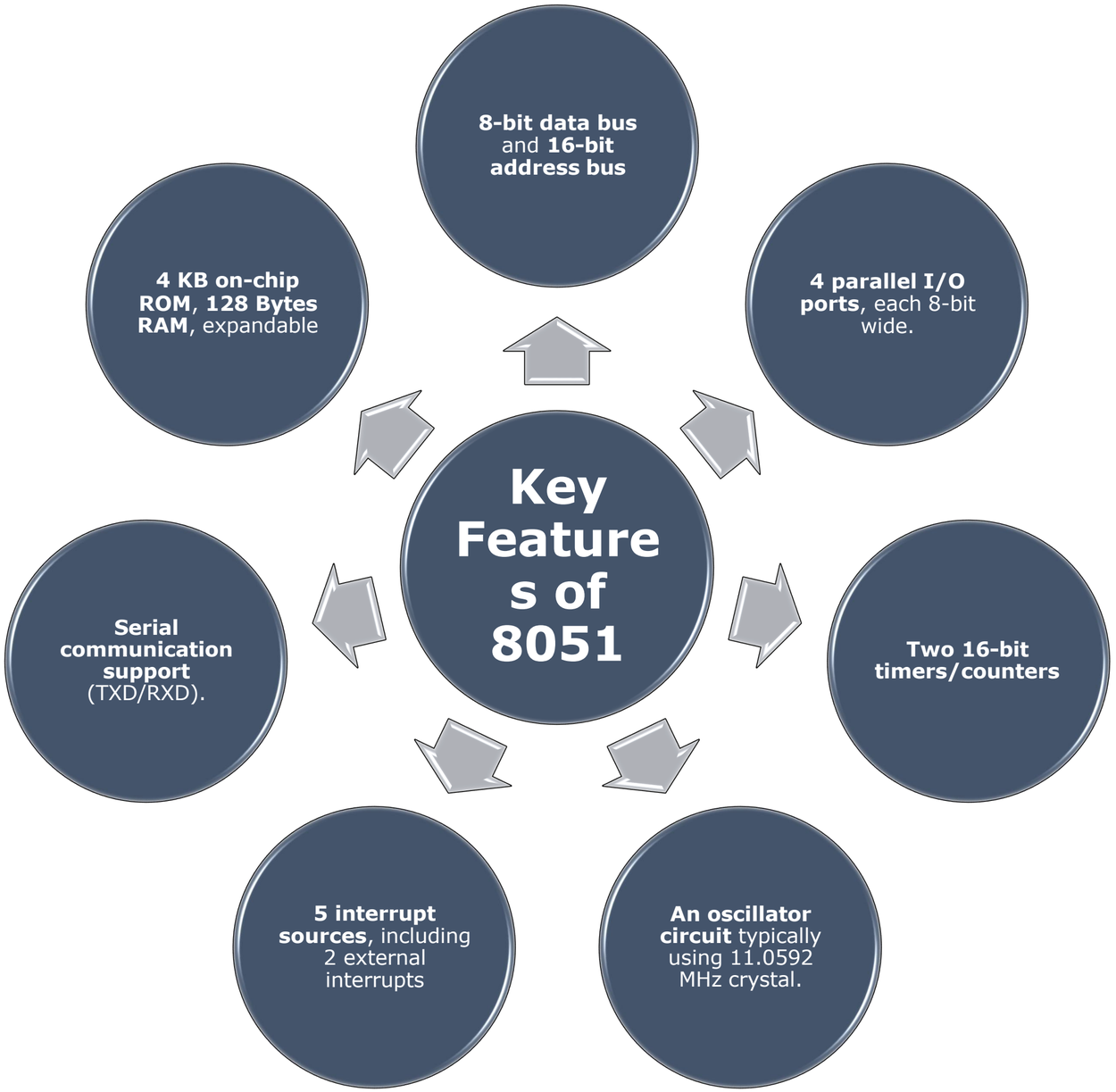
Automobiles

- Controls engine functions in hybrid vehicles
- Powers cruise control and anti-lock braking systems
- Offers efficiency and reliability

Medical Devices

- Embedded in devices like glucose and blood pressure monitors
- Displays accurate readings
- Ensures dependable healthcare diagnostics

Key Features of 8051 Microcontroller



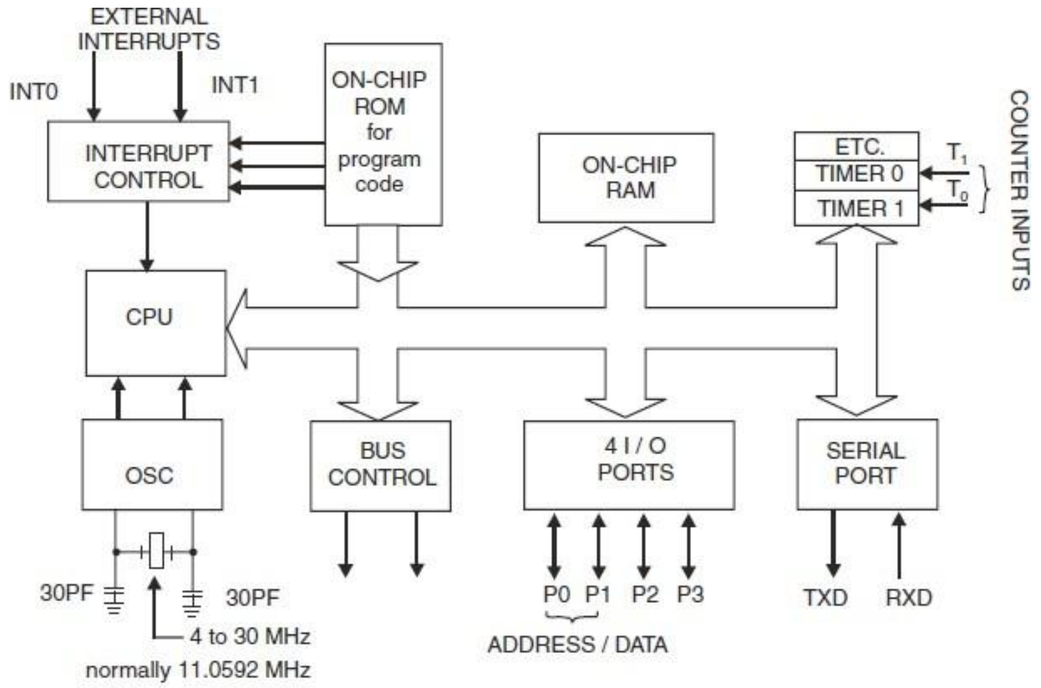


Fig. 1.2 8051 architecture

Pin Diagram of 8051

Port 0 (P0.0 – P0.7) Pins 32 to 39

Works as input/output

Also used for **address/data** in external memory mode

Needs **external pull-up resistors** when used as I/O

Port 1 (P1.0 – P1.7) Pins 1 to 8

Simple input/output port

No alternate functions

Has **internal pull-up resistors**

Port 2 (P2.0 – P2.7) Pins 21 to 28

Works as input/output

Used for **higher address lines** (A8–A15) in external memory mode

Has **internal pull-up resistors**

1.5 PIN DIAGRAM OF 8051

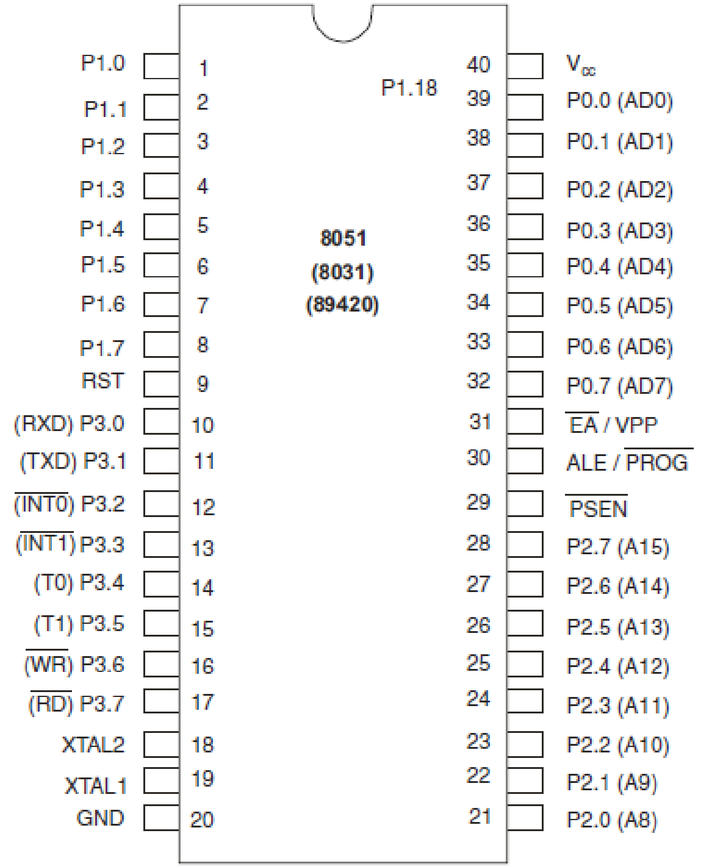


Fig. 1.3 Pin diagram of MC 8051

Port 3 (P3.0 – P3.7) – Simple Description Pins 10 to 17

Port 3 is an **8-bit input/output port**

Each pin also has a **special alternate function**

Commonly used for **interrupts**, **serial communication**, **timers**, and **external memory control**

Has **internal pull-up resistors**

Like other ports, it's also **bit-addressable** (each pin can be controlled individually)

Pin	P3.x	Alternate Function	Purpose
10	P3.0	RXD	Serial data input
11	P3.1	TXD	Serial data output
12	P3.2	INT0	External interrupt 0
13	P3.3	INT1	External interrupt 1
14	P3.4	T0	Timer 0 external input
15	P3.5	T1	Timer 1 external input
16	P3.6	WR	Write signal for external memory
17	P3.7	RD	Read signal for external memory

Port 3 (P3.0 – P3.7) – Simple Description Pins 10 to 17

Power and Clock Pins

Pin	Name	Function
40	Vcc	Power supply (+5V)
20	GND	Ground
19	XTAL1	Input to internal oscillator amplifier
18	XTAL2	Output from oscillator circuit

Control Pins for Memory Interfacing

Pin	Name	Function
9	RST	**Reset input** (active HIGH) – resets the microcontroller
31	EA/VPP	**External Access** – If low, MCU fetches code from external ROM
30	ALE	**Address Latch Enable** – Used to demultiplex address/data from Port 0
29	PSEN	**Program Store Enable** – Active LOW signal for reading external program memory

Architectural Block Diagram of 8051

- **Component of 8051**
- Arithmetic Logic Unit (ALU)
- Accumulator (Register A)
- Register B
- Data Pointer (DPTR)
- Program Counter (PC)
- Input Output ports
- Timer and Counter
- Serial port,Oscillator,Interrupt
- RAM
- ROM
- Program Status Word (PSW)
- Special Function Registers (SFR)

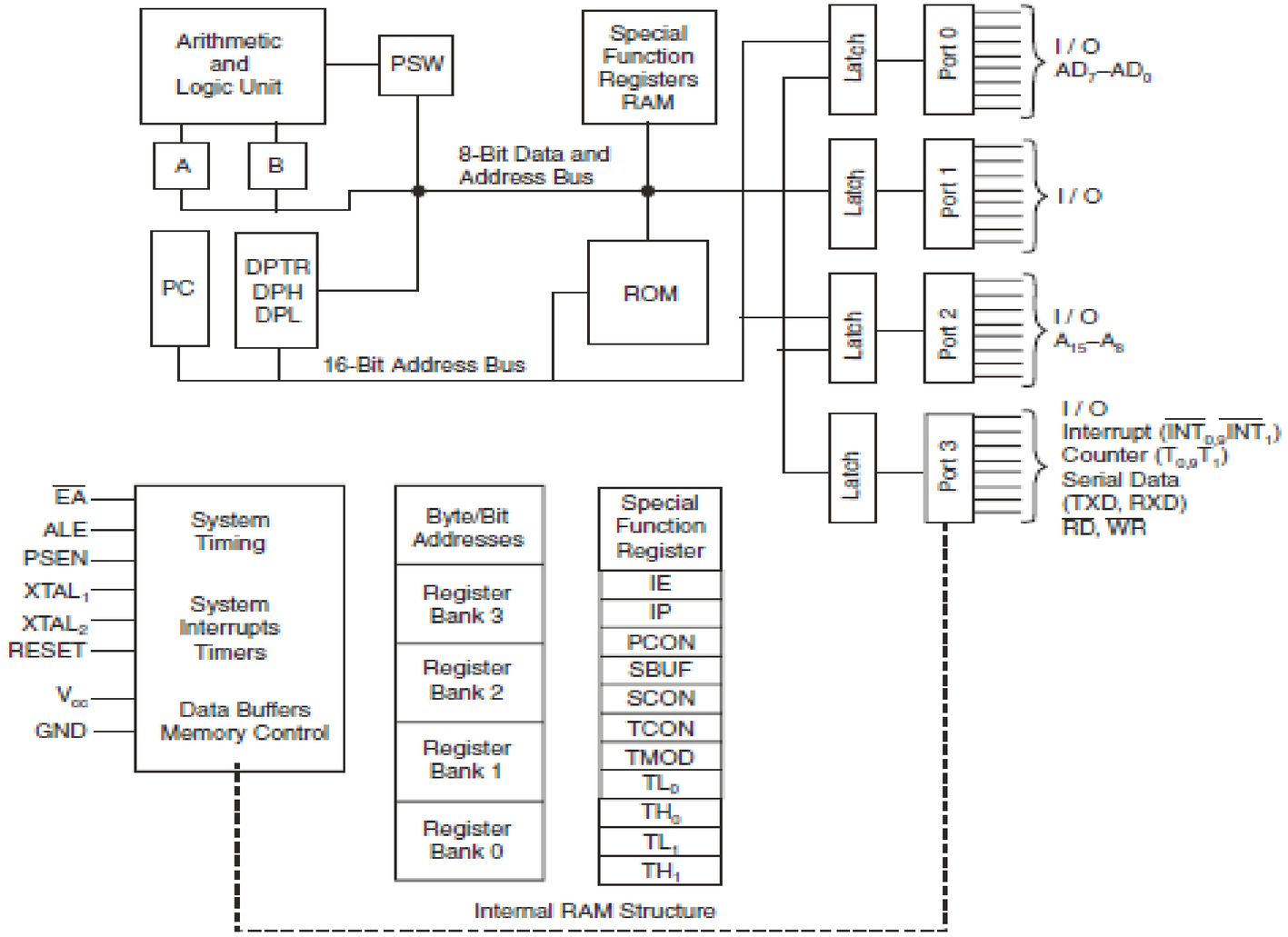


Fig. 1.4 Architectural block diagram of microcontroller 8051

8051 Components: ALU,A Register ,B Register

ALU – Arithmetic Logic Unit

- Performs all arithmetic (+, -, etc.) and logic (AND, OR, etc.) operations
- Works with **A (Accumulator)** and **B Register**

Accumulator (A Register)

- Most important 8-bit register
- Used in **all arithmetic/logic** operations
- **Bit-addressable** and part of SFRs

B Register

- 8-bit, used **only** in MUL AB and DIV AB instructions
- **Bit-addressable**, temporary storage during math

8051 Components: PC,DPTR

Program Counter (PC)

- 16-bit register that points to the **next instruction in ROM**
- Increments automatically after each instruction
- Range: **0000H to FFFFH** (up to 64KB code space)

Data Pointer (DPTR)

- 16-bit register used for **external memory access**
- Divided into:
  - **DPL (Low byte)**
  - **DPH (High byte)**
- Used in MOVX and MOVC instructions to fetch data from code or external memory

8051 Components: – Versatile Data Interfaces

4 Parallel I/O Ports:

- **P0, P1, P2, P3** – Each 8-bit wide = Total **32 I/O lines**
- Each pin is **bit-addressable** (can be individually set/cleared)

Registers Used:

- Each port has a dedicated **Special Function Register (SFR)**:
  - P0, P1, P2, P3

.

Port	Standard Use	Alternate Function
P0	I/O port	Multiplexed address/data bus (AD0–AD7)
P1	I/O only	— (no alternate function)
P2	I/O port	Higher address lines AD8–AD15
P3	I/O + Control signals	RXD, TXD, INT0/1, T0/1, WR, RD

8051 Components: Timers and Counters

Purpose:

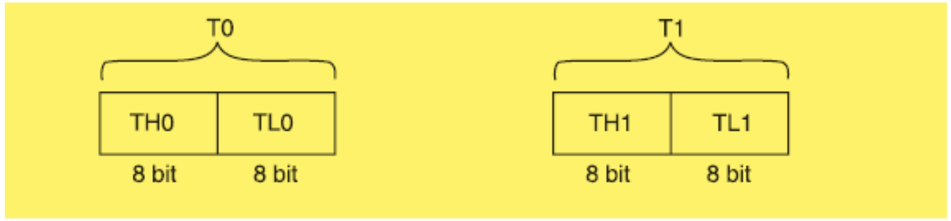
- Generate time delays (Timer Mode)
- Count external events (Counter Mode)

🔧 **Available Timers:**

- **Timer 0 (T0)**

Timer 1 (T1)

- Both are **16-bit** (range: 0000H–FFFFH)



Registers:

- TH0 / TL0, TH1 / TL1 – Hold timer values
- TMOD – Selects mode (Timer/Counter, 8/16-bit)
- TCON – Controls start/stop flags

Modes:

Timer Mode: Internal clock used for time delay

Counter Mode: External pulses counted on T0/T1 pins

Applications:

Delay generation, pulse counting, real-time scheduling

8051 Components: Serial port, Oscillator, Interrupt

Serial Port
Enables serial communication between the 8051 and external devices.

- **TXD (P3.1)**: Transmit data
- **RXD (P3.0)**: Receive data
- Controlled by SBUF, SCON
- Supports **4 modes** (synchronous/asynchronous)

Oscillator
Provides the clock signal that drives the microcontroller's operations.

Uses crystal via **XTAL1** & **XTAL2**

Typical frequency: **11.0592 MHz**

Determines **machine cycle speed**

Interrupts
Special signals that interrupt normal program flow to handle urgent tasks.

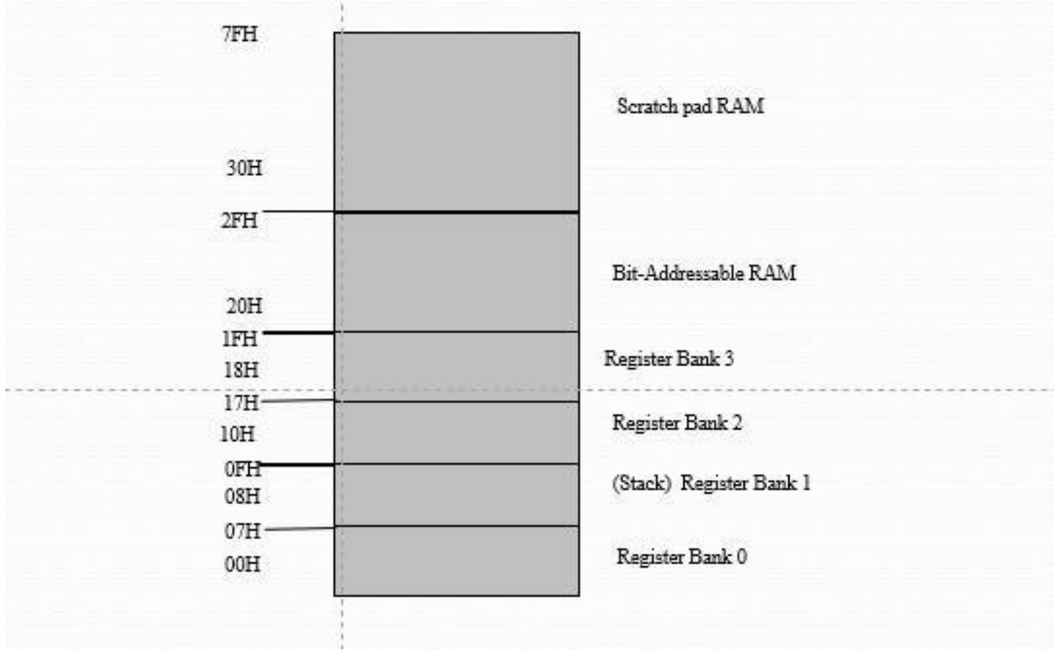
- **5 sources**: INT0, INT1, Timer0, Timer1, Serial
- Controlled by: IE (Enable), IP (Priority)

8051 Components: 128 Byte RAM Overview

Temporary **data storage** during execution

Total RAM:

- **128 bytes (00H–7FH)** user-accessible
- Divided into:
  - **Register banks (R0–R7)**
  - **Bit-addressable area**
  - **Scratchpad RAM**



Divided as:

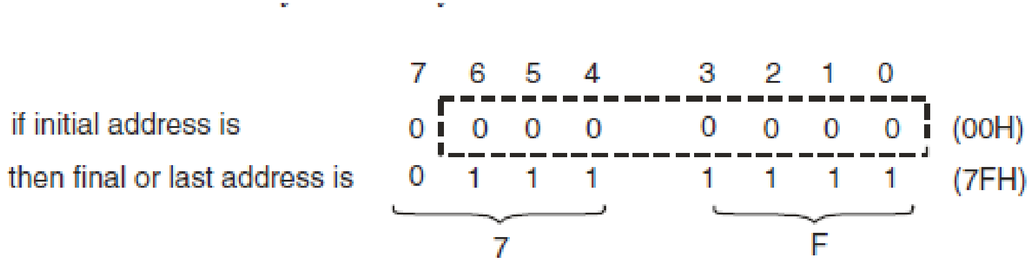
Lower 32 Bytes (00H–1FH): Register banks

Next 16 Bytes (20H–2FH): Bit-addressable RAM

Upper 80 Bytes (30H–7FH): General-purpose scratchpad RAM

Memory Mapping Range:

- Total = **128 locations** = **2^7 = 00H to 7FH**



8051 Components: 128 Byte RAM Overview

RAM Addressing and Structure

Register Banks:

4 banks, each with 8 registers (R0–R7)

Default bank: **Bank 0 (00H–07H)**

Others: Bank 1 (08H–0FH), Bank 2 (10H–17H), Bank 3 (18H–1FH)

Bit-Addressable Area (20H–2FH):

Allows manipulation of individual bits

Useful for flags and control applications

Scratch Pad RAM (30H-7F)

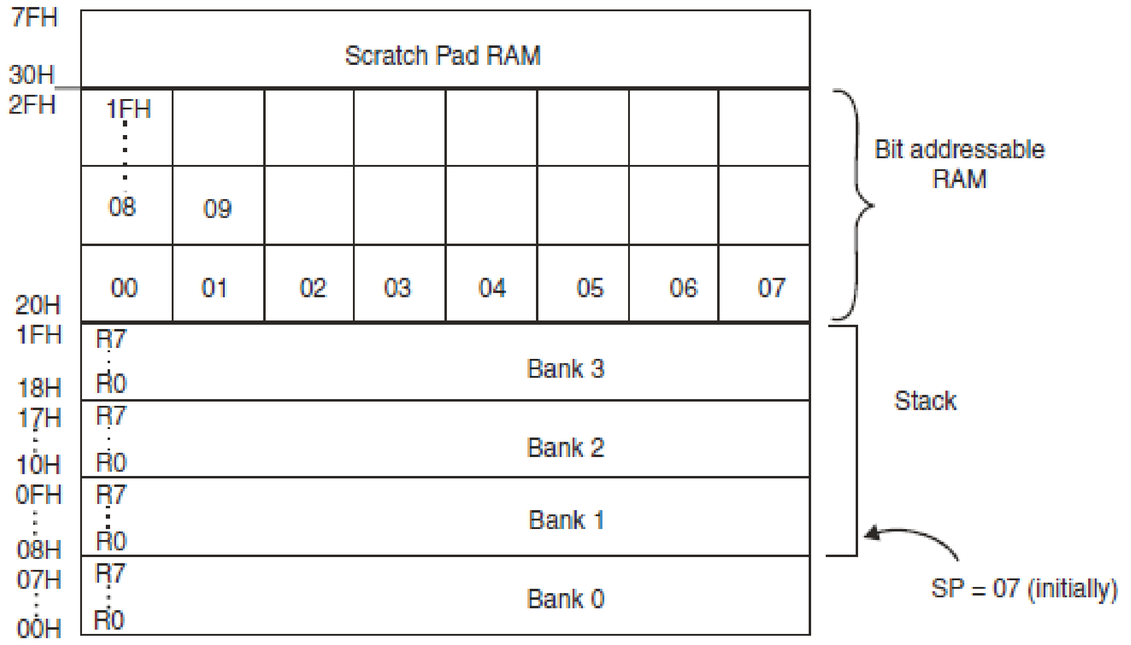


Fig. 1.6 Structure of RAM

Register Bank vs. Stack Conflict:

Problem: Stack (starts at 08H) overlaps with Register Banks 1–3 (08H–1FH) → memory conflict.

Effect: Data corruption and unpredictable behavior during execution.

Solution: Shift stack to safe area using MOV SP, #2FH → uses Scratchpad RAM (30H–7FH).

8051 Components: On-Chip ROM (Program Memory)

8051 - ROM Capacity:

- **4 KB** on-chip **Read-Only Memory**
- Used to store **program code** permanently
- **Non-volatile** – retains data after power off

Purpose:

- Holds fixed program logic/algorithms
- Cannot be modified during execution

Comparison with RAM		
Feature	ROM	RAM
Usage	Program code	Data storage
Volatility	Non-volatile	Volatile
Mutability	Fixed during run	Changeable

ROM Addressing:

Memory Address Range:

- 4 KB = 2^{12} = **0000H to 0FFFH**

Controlled by the **Program Counter (PC)**

PC is **16-bit**, can address up to **64 KB** (0000H–FFFFH)

External ROM Expansion:

Can interface **up to 64 KB** ROM externally

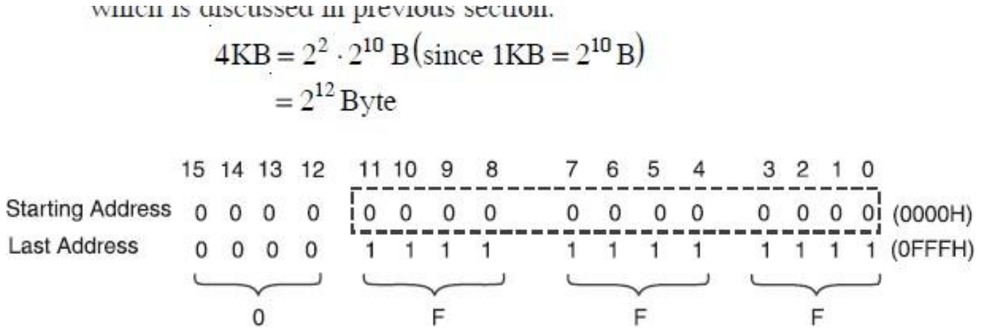
Useful for large applications

- **Example Calculation:**

4 KB = 2^{12} = 0000H to 0FFFH

8 KB = 2^{13} = 0000H to 1FFFH

64 KB = 2^{16} = 0000H to FFFFH



Address range of PC is 0000H to 0FFFH total 4KB locations are available for ROM

from 0000H to 0FFFH.

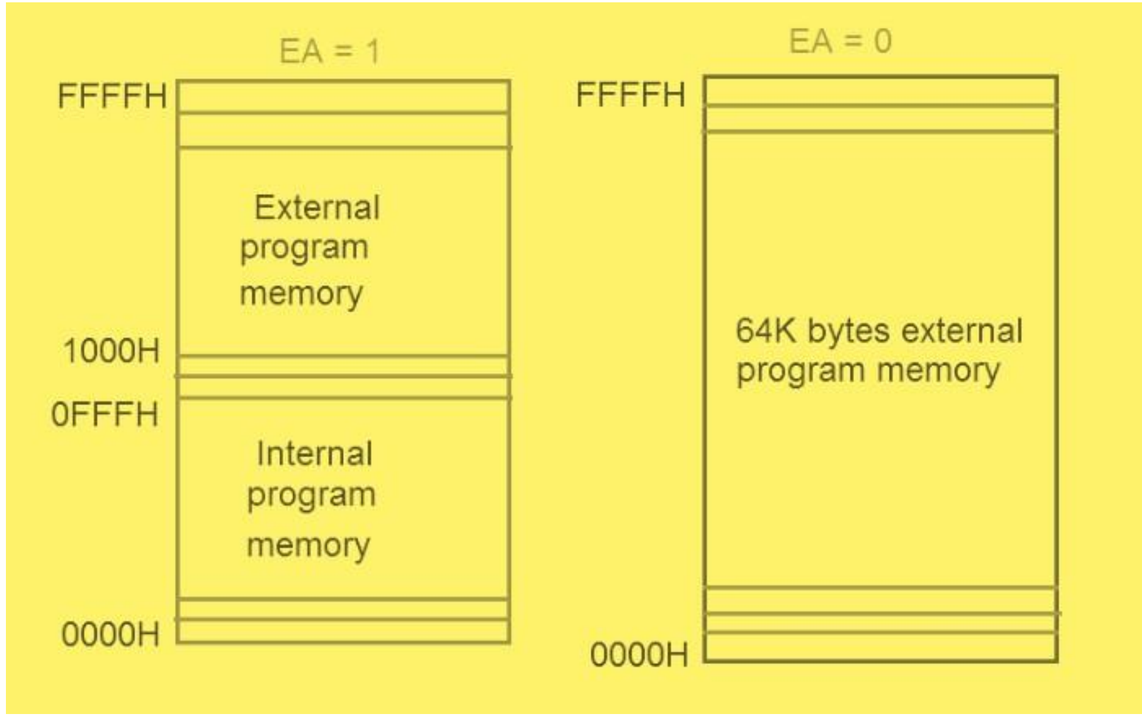
At which we can save the program.

EA Pin – ROM Access Control in 8051 Microcontroller

The **EA (External Access)** pin in the 8051 microcontroller determines whether the microcontroller will use its **internal ROM** or **external program memory** for fetching instructions.

•If **EA is high (connected to Vcc)**, the micr ocontroller **executes code from internal ROM**. This is the standard operating mode when the on-chip ROM is sufficient to hold the entire program.

•If **EA is low (connected to GND)**, the microcontroller **ignores internal ROM** and **fetches all instructions from external memory** This mode is used when larger program memory is required than what is available internally.



8051 Components: Flag Bits & PSW Register

What is PSW?

Program Status Word (PSW) is an 8-bit **Special Function Register**

Reflects **arithmetic status** of operations in the Accumulator (A)

Addressable and **bit-wise accessible**

- **PSW Register**

CY	AC	F0	RS_1	RS_0	OV	--	P

Carry flag	PSW.7	**CY**
Auxiliary carry flag	PSW.6	**AC**
Available to the user for general purpose	PSW.5	**--**
Register Bank selector bit 1	PSW.4	**RS1**
Register Bank selector bit 0	PSW.3	**RS0**
Overflow flag	PSW.2	**OV**
User define bit	PSW.1	**--**
Parity flag Set/Reset odd/even parity	PSW.0	**P**

RS 1	RS 0	Register Bank	Address
0	0	0	00H-07H
0	1	1	08H-0FH
1	0	2	10H-17H
1	1	3	18H-1FH

8051 Components: Special Function Registers (SFRs)

What are SFRs?

- Registers at addresses **80H to FFH** in internal RAM
- Control and monitor internal features: **I/O ports, timers, serial port, power, etc.**
- Some are **bit-addressable** for precise control

Why SFRs Matter:

- All hardware-level operations go through these registers
- Essential for tasks like **timer setup, serial communication, I/O control**

Special Function Registers

Name	Function	Name	Function
A	Accumulator	SBUF	Serial Port data buffer
B	Arithmetic	SP	Stack Pointer
DPH	Addressing Ext Memory	TMOD	Timer/Counter mode cntrl
DPL	Addressing Ext Memory	TCON	Timer/Counter cntrl
IE	Interrupt enable	TL0	Timer0 lower byte
IP	Interrupt Priority	TH0	Timer0 higher byte
P0	I/O Port Latch	TL1	Timer1 lower byte
P1	I/O Port Latch	TH1	Timer1 higher byte
P2	I/O Port Latch		
P3	I/O Port Latch		
PCON	Power Control		
PSW	Pgm Status Word		
SCON	Serial PortCntrl		

Assembly Language Basics

What is Assembly Language?

- A **low-level language** that uses **mnemonics** instead of binary
- Easier than machine code, but gives **full control** of hardware
- Specific to the processor (8051 in this case)

Instruction Format: [Label:] Mnemonic [Operands] [;Comment]

- **Label**: Optional name for a line (used for jumps/loops)
- **Mnemonic**: The instruction (e.g., MOV, ADD, SJMP)
- **Operand**: Data or registers the instruction works on
- **Comment**: Starts with ;, explains the code

Example:

```
Loop:  MOV R1, #25H   ; Move 25H to R1
       ADD A, R1      ; Add R1 to Accumulator
       SJMP Loop      ; Repeat (jump back to Loop)
```

Addressing Modes in 8051 Microcontroller

Definition: Addressing modes define how the microcontroller accesses data for operations.

1. Immediate Addressing

- Operand is specified in the instruction itself.
- Example: MOV A, #30H

2. Register Addressing

- Data is in registers R0 to R7.
- Example: MOV A, R1

3. Direct Addressing

- Access internal RAM or SFRs by their direct address.
- Example: MOV A, 60H

4. Indirect Addressing

- Register holds the memory address (R0, R1, or DPTR).
- Example: MOV A, @R0

5. Indexed Addressing

- Base address (DPTR or PC) + offset (A). Used in lookup tables.
- Example: MOVC A, @A+DPTR

6. Relative Addressing

- Used with conditional jumps; offset added to PC.
- Example: SJMP LABEL

Arithmetic Instructions in 8051

1. Addition (ADD, ADDC)

- ADD A, #data : Add immediate to A
- ADD A, Rn : Add register to A
- ADD A, direct : Add memory to A
- ADDC is used to include carry in addition.
- Affects CY, AC, OV flags.

2. Multiplication (MUL)

- MUL AB : Multiplies A and B (8-bit each)
- Result: Lower byte in A, higher byte in B

3. Division (DIV)

- DIV AB : Divides A by B (unsigned)
- Quotient in A, Remainder in B

Note: Flags CY and OV are affected in arithmetic operations

Example:Addition (ADD, ADDC)

MOV A, #25H
ADD A, #10H ; A = 35H

Add the contents of A with register Rn with or without ca rry.

iADD A, R5

Add the contents of A with contents of memory with or w ithout carry using direct and indirect addressing

i. ADD A, 51H

ii ADDC A, 75H

iii. ADD A, @R1

Example:Multiplication (MUL)

MOV A, #12H
MOV B, #10H
MUL AB ; A = 20H, B = 01H

Example:Division (DIV)

MOV A, #20H
MOV B, #05H
DIV AB ; A = 04H, B = 00H

Branch Instructions in 8051

1. Unconditional Branch

- Transfer control without any condition.
- **LJMP address :** Long jump (16-bit address)
- **AJMP address** : Absolute jump (11-bit address within 2KB)
- **SJMP rel** : Short jump (relative jump within −128 to +127)

3. Bit-Level Branch

- Operate on bit values.
- **JB bit, rel** : Jump if bit = 1
- **JNB bit, rel** : Jump if bit = 0
- **JBC bit, rel :** Jump if bit = 1 and clear it

2. Conditional Branch

- Control is transferred only if condition is true.
- **JZ rel** : Jump if A = 0
- **JNZ rel** : Jump if A ≠ 0
- **JC rel** : Jump if carry = 1
- **JNC rel** : Jump if carry = 0
- **CJNE A, #data, rel** : Jump if A ≠ data
- **DJNZ Rn, rel :** Decrement Rn, jump if not zero

Assembler Directives in 8051

1. ORG (Origin)

Sets the starting address for code/data.

Example: ORG 0000H

2. EQU (Equate)

Defines a constant without using memory.

Example: N EQU 10

3. SET

Similar to EQU, but allows redefinition.

Example: COUNT SET 20

4. DB (Define Byte)

Reserves memory and sets 8-bit values.

Example: DATA DB 0FFH

5. END

Marks the end of the source code file.

Example: END

Assembly language programs

Program 1:Problem Statement:
Write a program to add the values of locations 50H and 51H and store the result in locations in 52h and 53H**.**

```
ORG 0000H              ; Set program counter to 0000H

MOV A, 50H             ; Load contents of memory location 50H into A
ADD A, 51H             ; Add contents of 51H to A
MOV 52H, A             ; Store the LSB of result in 52H

MOV A, #00             ; Load 00H into A to prepare for carry
ADDC A, #00            ; Add carry to A
MOV 53H, A             ; Store MSB (carry) in 53H

END                    ; End of program
```

Program 2:
Write a program Store FFH into memory locations 50H to 58H using indirect addressing mode and a loop.

```
ORG 0000H

MOV A, #0FFH           ; Load FFH into accumulator A
MOV R0, #50H           ; Initialize pointer R0 to 50H
MOV R5, #08H           ; Loop counter for 8 bytes

START:
MOV @R0, A             ; Store FFH at address pointed by R0
INC R0                 ; Move to next address
DJNZ R5, START         ; Decrement counter and repeat if not zero

END
```

Assembly language programs

Program 3

Write a program to Calculate the sum of numbers from 1 to N (N = 15), and store the result at 70H

```
ORG 0000H
N EQU 15              ; Constant N = 15

MOV R0, #00           ; Initialize counter
CLR A                 ; Clear accumulator for sum

AGAIN:
INC R0                ; Increment counter
ADD A, R0             ; Add current count to
accumulator
CJNE R0, #N, AGAIN    ; Continue until R0 == 15

MOV 70H, A            ; Store final result in 70H

END
```

Program 4:

Write a program to Multiply two 8-bit numbers stored at 70H and 71H. Store result in 52H (LSB) and 53H (MSB)

```
ORG 0000H

MOV A, 70H            ; Load multiplicand into A
MOV B, 71H            ; Load multiplier into B
MUL AB                ; Multiply A and B

MOV 52H, A            ; Store LSB of result in 52H
MOV 53H, B            ; Store MSB of result in 53H

END
```

.

Assembly language programs

Program 5:

Write a program to Find the average of five 8-bit numbers (located at 40H to 44H). Store the result in 55H.

```
ORG 0000H

MOV 40H, #05H       ; First number
MOV 41H, #55H       ; Second number
MOV 42H, #06H       ; Third number
MOV 43H, #1AH       ; Fourth number
MOV 44H, #09H       ; Fifth number

MOV R0, #40H        ; Set pointer
MOV R5, #05H        ; Loop counter
MOV B, R5           ; Copy divisor into B
CLR A               ; Clear accumulator

LOOP:
ADD A, @R0          ; Add value to A
INC R0
DJNZ R5, LOOP

DIV AB              ; Divide total by 5
MOV 55H, A          ; Store quotient (average)

END
```

Interfacing Seven Segment Display with 8051

Introduction to What is a Seven Segment Display?

- Electronic display device for showing decimal numerals and some characters.
- Contains **7 LEDs** arranged in shape of figure ‘8’ + 1 LED for decimal point (dot).

Types:

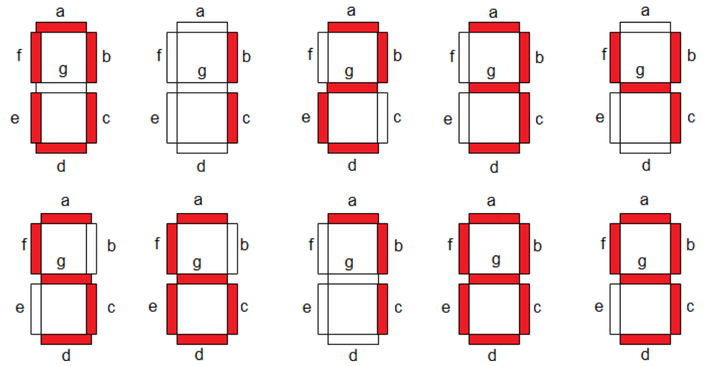
- **Common Cathode (CC):** All cathodes tied together, individual anodes.
- **Common Anode (CA):** All anodes tied together, individual cathodes.

Pin Configuration & Display Patterns

Pin Configuration:

- Pins labeled as: **a, b, c, d, e, f, g** (segments), and **h/dp** (dot point).
- Selection of segments forms numbers or characters.

Digit	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1
…							
9	1	1	1	1	0	1	1



Interfacing Seven Segment Display with 8051

Working Principle & Circuit Description

How It Works:

1.Select LEDs (segments) to **form a digit** by giving high/low signals.

2. **Digit Drive Pattern:** A binary/hex code sent to the microcontroller's port to light up corresponding segments.
3. **Look-Up Table (LUT):** Holds patterns for digits **0-9**.
4. **Program uses MOVC A, @A+PC** to fetch required pattern from LUT.

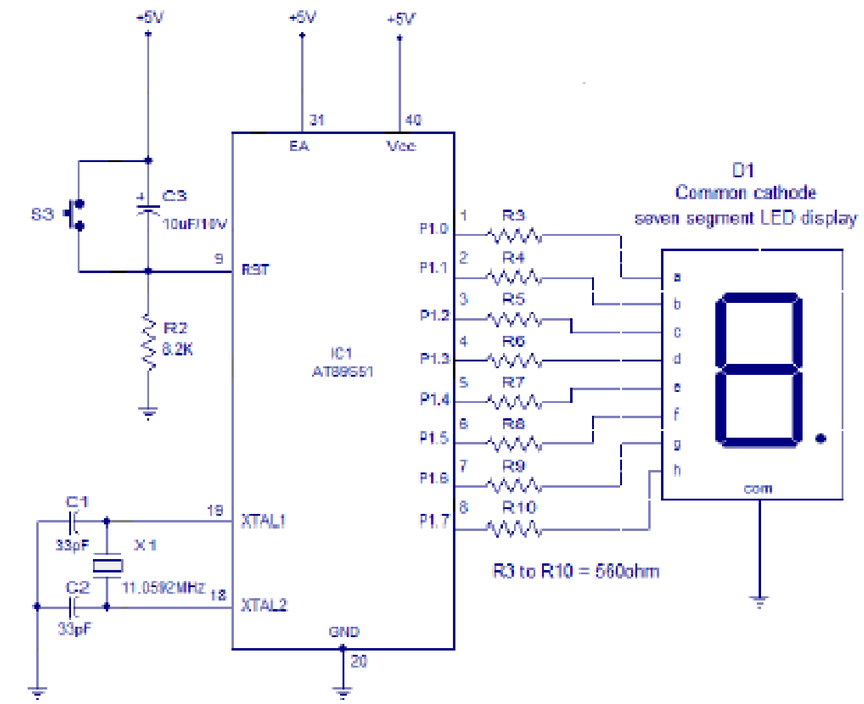
Circuit Diagram

Basic Setup

- **Microcontroller Used:** AT89S51
- **Seven Segment Connected to:** Port 1
- **Components Used:**
  - **R3 - R10:** Current limiting resistors
  - **S3, R2, C3:** Reset + debouncing circuitry
  - **C1, C2, X1:** Oscillator circuit

Purpose:

- Display numbers 0 to 9 repeatedly.



Interfacing 7 segment display to 8051

Interfacing Seven Segment Display with 8051

Code:

```
ORG 000H         ; Starting address of program memory
START: MOV A,#00001001B   ; Initial value in accumulator (9 decimal)
       MOV B,A           ; Copy A to B for later use
       MOV R0,#0AH       ; Counter R0 initialized to 10 (0AH)

LABEL: MOV A,B           ; Load A with value of B
       INC A             ; Increment A by 1 for next pattern address
       MOV B,A           ; Store back in B
       MOVC A,@A+PC      ; Fetch digit drive pattern from Look-Up Table (LUT)
       MOV P1,A          ; Output pattern to Port 1
       ACALL DELAY       ; Call delay subroutine for visible effect
       DEC R0            ; Decrement counter R0
       MOV A,R0
       JZ START          ; If R0=0, restart count from 0
       SJMP LABEL        ; Otherwise, continue looping

; Look Up Table for digits 0 to 9 (common cathode patterns)
       DB 3FH  ; 0
       DB 06H  ; 1
       DB 5BH  ; 2
       DB 4FH  ; 3
       DB 66H  ; 4
       DB 6DH  ; 5
       DB 7DH  ; 6
       DB 07H  ; 7
       DB 7FH  ; 8
       DB 6FH  ; 9

; Delay Subroutine
DELAY: MOV R4,#05H
WAIT1: MOV R3,#00H
WAIT2: MOV R2,#00H
WAIT3: DJNZ R2,WAIT3
       DJNZ R3,WAIT2
       DJNZ R4,WAIT1
       RET
END
```

Summary & Key Points

Program Highlights:

- Uses **MOVC A,@A+PC** to fetch patterns from **LUT**.
- **Counter (R0)** ensures **0 to 9** display repeatedly.
- LUT simplifies pattern generation.
- Can be extended to **multiple displays** for larger numbers.

Applications:

- **Electronic counters**
- **Digital clocks**
- **Embedded calculators**

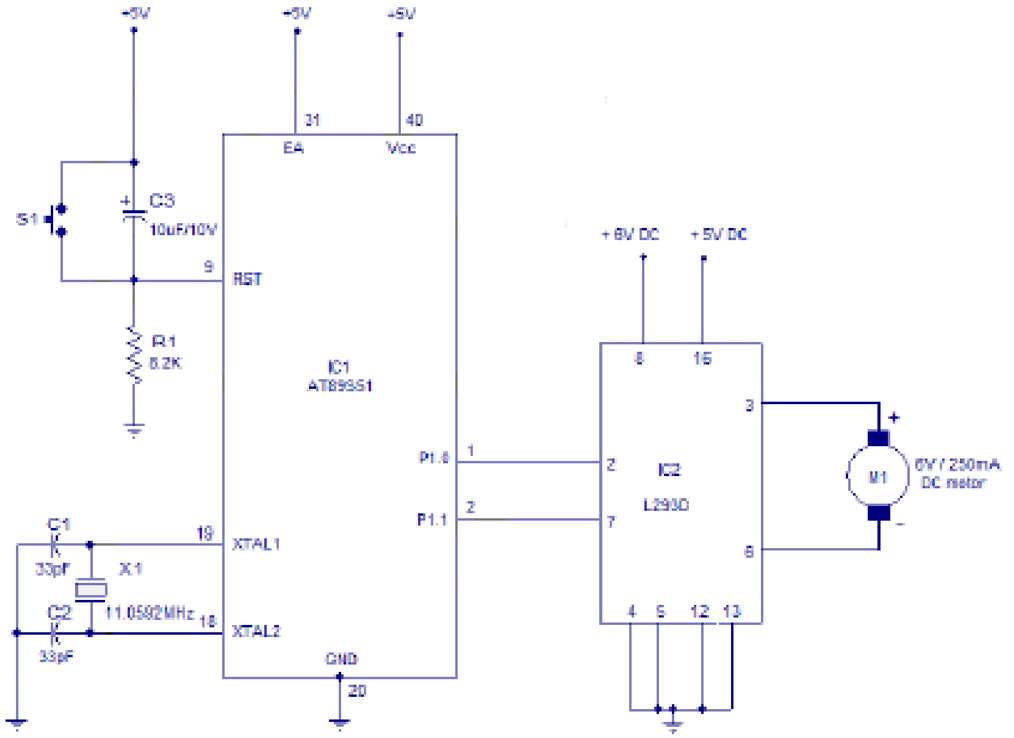
Interfacing DC Motor with 8051 Microcontroller

Introduction & Purpose

- **Goal:** Control the speed and direction of a DC motor using the 8051 microcontroller.
- **Need for Interface:** 8051 cannot supply sufficient current to drive the motor directly, hence a motor driver IC (L293D) is used.

Motor Driver (L293D) & Working Principle

- **L293D Features:**
  - 16-pin IC, controls 2 DC motors.
  - 600mA/channel, voltage range 4.5V–36V.
  - Built-in protection with clamp diodes.
  - Works on H-Bridge concept to control motor direction by changing current flow.



- **Circuit Overview:**
  - AT89S51 (8051) controls L293D via P1.0 & P1.1.
  - Motor connected across output pins 3 & 6 of L293D.
  - Direction is controlled by logic combinations on P1.0 and P1.1.

Interfacing DC Motor with 8051 Microcontroller

Program to Control Motor

```
ORG 00H                ; Program starts here
MAIN: MOV P1,#00000001B    ; Motor rotates clockwise
      ACALL DELAY
      MOV P1,#00000010B    ; Motor rotates anticlockwise
      ACALL DELAY
      SJMP MAIN            ; Repeat cycle forever

DELAY: MOV R4,#0FH
WAIT1: MOV R3,#00H
WAIT2: MOV R2,#00H
WAIT3: DJNZ R2,WAIT3
       DJNZ R3,WAIT2
       DJNZ R4,WAIT1
       RET
END
```

Logic:

- **P1.0 = 1, P1.1 = 0 → Clockwise**
- **P1.0 = 0, P1.1 = 1 → Anticlockwise**

Summary

- **Motor Direction:** Controlled by **P1.0 & P1.1** logic.
- **Current Limit:** ≤ **600mA** per channel.
- **Applications:** Robotics, automation, motorized embedded systems.
- **Caution:** Ensure motor voltage and current are within L293D’s specifications.